

LAB EVALUATION MANUAL

Experiment: Build, Deploy and Analyze a Containerized Microservice Application Under Varying Workloads

Aim

To develop a microservice-based application containing three independent services, containerize and deploy the services using Docker, establish inter-service communication, generate varying workloads, monitor resource utilization, and analyze application performance.

General Instructions

- Students may choose any suitable application domain such as Online Shopping, Food Delivery, Hospital Management, Library Management, Student Management, or another approved domain.
- The application must contain exactly three independent microservices.
- Each microservice must have a clear responsibility and at least one working REST API endpoint.
- Docker must be used to containerize the services, and Docker Compose must be used for deployment.
- Students must perform workload testing and use actual measured values for their analysis.
- Each checkpoint must be demonstrated to the evaluator before proceeding to the next checkpoint.

Minimum Architecture

Client → Service 1 → Service 2 / Service 3

The exact responsibilities and communication flow may differ according to the selected application, but all three services must communicate through the container network where required.

Checkpoint 1 — Design and Develop the Microservices

1. Select an application domain.
2. Identify three independent microservices.
3. Define the responsibility of each service.
4. Implement the three services using a suitable programming language/framework.
5. Create REST API endpoints for each service.
6. Run and test each service independently.
7. Verify that the APIs return the expected responses.

Completion condition: All three microservices are independently working and their APIs can be demonstrated.

Checkpoint 2 — Containerize and Deploy the Application

1. Create a separate Dockerfile for each microservice.
2. Create the required dependency/configuration files.
3. Build a Docker image for each microservice.
4. Verify that all three images are available using Docker commands.
5. Create a docker-compose.yml file.
6. Configure all three services in Docker Compose.

7. Deploy the complete application using Docker Compose.

8. Verify that all three containers are running.

Completion condition: Three Docker images are built successfully and all three microservices are running as containers.

Checkpoint 3 — Establish and Demonstrate Microservice Communication

1. Create/configure the Docker network through Docker Compose.

2. Connect all three services to the same application network.

3. Configure service-to-service communication using Docker service names.

4. Test communication between the microservices.

5. Perform an end-to-end request involving multiple services.

6. Verify that the final response is successfully returned to the client.

Completion condition: Students demonstrate successful communication between the three microservices and an end-to-end request.

Checkpoint 4 — Generate Varying Workloads and Monitor Performance

1. Select an API suitable for workload testing.

2. Use a load-testing tool or create a suitable workload generator.

3. Test the application using at least five workload levels.

4. A suggested workload is 1, 2, 4, 8, and 16 concurrent requests.

5. Record response time and throughput for every workload.

6. Monitor all three containers using docker stats or another suitable monitoring method.

7. Record CPU and memory utilization.

8. Record successful and failed requests, if applicable.

Completion condition: Performance measurements are collected for all workload levels.

Checkpoint 5 — Analyze and Present the Results

1. Prepare an observation table containing workload, concurrency, response time, throughput, failed requests, CPU utilization, and memory utilization.

2. Create suitable graphs from the measured results.

3. Compare application performance at different workload levels.

4. Identify how increasing workload affects the application.

5. Identify which microservice consumes more resources.

6. Explain any performance degradation or failures observed.

7. Demonstrate the complete application to the evaluator.

8. Explain the architecture, containerization, communication, workload testing, and performance results.

Completion condition: Students demonstrate the complete experiment and explain their measured results.

Suggested Workload Levels

Test	Concurrent Requests
W1	1
W2	2
W3	4
W4	8
W5	16

Suggested Observation Table

Workload	Concurrency	Response Time	Throughput	Failed	CPU	Memory
W1	1					
W2	2					
W3	4					
W4	8					
W5	16					

Recommended Graphs

- Concurrent Requests vs Average Response Time
- Concurrent Requests vs Throughput
- Concurrent Requests vs CPU Utilization
- Concurrent Requests vs Memory Utilization

Final Evaluation — 5 Marks

Checkpoint	Evaluation Area	Marks
1	Design and develop 3 microservices	1
2	Containerize and deploy using Docker	1
3	Establish and demonstrate inter-service communication	1
4	Generate varying workloads and monitor performance	1
5	Analyze results and demonstrate the complete experiment	1
	TOTAL	5

Final Student Deliverables

- Source code of the three microservices
- Three Dockerfiles
- docker-compose.yml
- Running Docker containers
- Demonstration of inter-service communication
- Workload test results
- CPU and memory observations
- Performance observation table
- Performance graphs
- Brief analysis and conclusion

Final Workflow

DEVELOP → CONTAINERIZE → DEPLOY → CONNECT → LOAD TEST → MONITOR → ANALYZE → DEMONSTRATE